

Eliminating Timing Channels in Microkernels

MSc Project Description

Simon Vinding Brodersen(rzn875)

May 11, 2026

Abstract

Timing channels present a significant security threat in modern computing, allowing unauthorized information flow through observable variations in execution time. Research has been made in creating secure microkernels designed to mitigate these channels from the ground up[1], [2], while other research focuses on closing timing channels at the user level[3]. It remains an open challenge to gather these findings for a more general and mainstream OS, and this thesis aims to investigate the feasibility of integrating time protection mechanisms into the widely used FreeRTOS kernel.

FreeRTOS provides a default priority-based preemptive scheduler, which is inherently vulnerable to timing channels. To address this, the project implements a **domain-level** scheduler that partitions execution into fixed time slices, ensuring that the execution of tasks in one domain does not affect the timing observed by another. Key to this implementation is the temporal fence (fence.t) instruction during domain switches to clear shared microarchitectural states[4], [5]. Further, constant-time domain creation mechanisms are introduced, allowing dynamic resource reconfiguration without introducing new timing channels.

The implementation was evaluated using QEMU system emulation. The results demonstrate that the modified kernel achieves constant-time domain scheduling with negligible variance in regard to instruction counting. While evaluation on true hardware remains undone, this work provides a foundation towards bringing timing protection to mainstream embedded systems.

Contents

1	Introduction	3
2	Background	4
2.1	Side-channels	4
2.2	Covert channels	5
2.3	Isolation	6
2.4	User-level mitigations	7
3	Related Work	8
3.1	The missing OS Abstraction: seL4	8
3.1.1	Threat model	8
3.1.2	Channels	8
3.1.3	Cloning the OS	9
3.1.4	Cache coloring	9
3.2	temporal_fence_t	9
3.2.1	With software support	10
3.3	S3K	10
3.3.1	Implementation	11
3.3.2	Time protection	12
3.3.3	Evaluation	12
3.4	FreeRTOS	13
4	Scheduling	14
4.1	The FreeRTOS scheduler	14
4.2	Closing Scheduling Based Timing Channels	14
4.3	Implementation	17
4.3.1	Extending the api	17
4.3.2	New configuration flags	19
4.3.3	Domain switching	19
4.3.4	Constant time domain creation	22
4.3.5	Per domain scheduling	25
4.3.6	Cross domain communication	26
5	Evaluation	28
5.1	The blinky example	28
5.2	Constant time	29
5.3	QEMU cycle accuracy	31

5.4	Limitations	31
5.5	The FreeRTOS kernel is vast	32
6	Discussion	33
6.1	Partitioned hardware	33
6.2	External interrupts	33
6.3	Out of order execution	33
6.4	Call for safe hardware interface	33
7	Conclusion	33

1 Introduction

Information channels are methods of communication in an unintended manner. One such example is timing channels, which occur when a tasks execution time varies based on the execution of another task in the system. Whether inadvertently, i.e. a side-channel, or purposely the other task is giving information to the current task by altering its execution time. This way timing channels bypass the operating system’s (OS’s) security enforcement leaking information through timing of observable events. At first many attacks were targeting different cryptographic libraries and leaking information about secret keys through these timing channels [6], [7]. Further, attacks have shown that it is possible to gather information across VMs running in a cloud computing setting [8]. Not only that, side-effects from out-of-order executions on modern processors gave way for the meltdown attacks to read arbitrary kernel-memory locations including personal data and passwords [9]. Different mitigations are moving forward, such as hiding kernel memory from user space tasks [10] to prevent Meltdown attacks, and a move to more constant time programming in cryptographic libraries with help from DSL languages such as FaCT [3] that aims to guarantee constant-time execution with respect to secretly marked information. While these mitigations do prove valuable, more modern CPU’s speculative execution still leaves doors open for more sophisticated attacks such as the ones seen in [11].

The attack front is clear, but the solution is not as simple. While some proposed solutions focus on the user side implementing secure programs, this creates a huge burden on the developer to implement secure code. Instead implementations of secure microkernels [1], [2] attempt to guarantee security through the OS. The operating system would then be in charge of partitioning domains in regards to the microarchitectural state, such that no channel between domains is possible. This is already done for memory, and examples of formally proven memory safe kernels

are available[12]. Attempts to create similar proof against the presence of timing channels have been carried out [13], but as far as this paper is aware no kernel is formally verified against timing channels.

Many of these attempts are focussed on the creation of a highly secure OS from the ground up, providing timing protection as a foundation of the OS. While this is a more robust solution for a truly safe OS the authors of [1] hint that "there is no reason time protection cannot be implemented in other systems...", something which could bring timing protection to more main stream applications and something this thesis will attempt to investigate further.

2 Background

2.1 Side-channels

Side-channels are unintended information channels, that leak information about the process. These channels infer secret data often through observable execution time of a process. The general case of such a channel can be seen in the snippet below:

```
1: if secret then  
2:   Some long computation  
3: else  
4:   Do nothing  
5: end if
```

This algorithm is a naive example, where we simply branch on some secret, and the execution time of the program. If for example this algorithm was part of some library, then an attacker could infer information about the secret by simply measuring the execution time of the call to the algorithm. This example is quite naive, and is one of the side channels constant time programming at the user level can prevent, such that we might do an equally long computation in both of the branches of the if statement. However, this still comes with some precautions, because even though the original program might seem sensibly constant time, some compilers might try to optimise the code, which could introduce new side-channel attacks such as the one seen in [14].

Further, more complex examples of side channels includes those specified within [11], where speculative execution can be used to leak information about an otherwise safe algorithm. An example from the paper is the following:

```

1: if  $x < \text{array1\_size}$  then
2:    $y = \text{array2}[\text{array1}[x] * 4096]$ 
3: end if

```

This function could again be part of some library or system call, which receives an unsigned integer x from the user. The process, which runs this library code or system call has access to both `array1` and `array2`. It does a bounds check on x , to make sure we have no out of bounds access, which could trigger an exception or access sensitive memory. If this algorithm was called multiple times, with correct input, i.e. where $x < \text{array1_size}$, then an attacker could train the branch predictor on modern CPUs making the CPU speculatively execute the expected path, which could bring in the location read from `array2` into the cache, and the attacker could then use attacks such as Flush+Reload or Prime+Probe to reveal which part of `array2` was loaded into the cache. With this, it can gather the index of the loaded item, and can figure out what was read from `array1[x]`, which could be some arbitrary secret that was obtained by the out of bounds indexing that was made possible by the branch predictor. A more detailed explanation along with a proof-of-concept is provided in [11], but this small example shows how seemingly safe code, might still leak secret information.

A multitude of such attacks, which make use of these side-channels are presented in [15], [16].

2.2 Covert channels

While side-channels are passive and exploit unintended leakage, covert channels involve two processes working together. This usually revolves around a sender (Trojan) and a receiver (spy), where the sender has access to high security data but is restricted in its communication with the receiver. To bypass this, the sender could make use of some resource which it shares with the receiver. This could be anything from microarchitectural state to some higher level cache, which is not partitioned. For example, the Trojan could use the shared resource to indicate a "1" and do nothing to indicate a "0". When the spy runs, it could measure the execution time of a cache access on the shared resource, and depending on how long this access takes gather if the Trojan was sending a "1" where a cache hit would occur or a "0" if a cache miss occurred.

Generally two main categories of covert channels exist; **Storage channels**: which modify some stored object, e.g. a file or some metadata; **Timing channels**: which change the time of different observable events for the receiver. For this thesis storage

channels are not referenced, as kernels such as seL4 have already formally proven the absence of them [12].

Covert timing channels work through the same mechanisms as the aforementioned side channel, but pose as a worst case, where the sender can make use of any and all leakage methods to convey the information. This way, we are not limited to the visible gadgets that might exist in a vulnerable program, but the information might also be conveyed through repeated use of non-constant time system calls, that depend on some shared resource, or as the aforementioned example shared use of the cache.

2.3 Isolation

Throughout many timing channel mitigations, isolation is a common factor. Generally there exists two types of isolation, namely **spatial** and **temporal** isolation. Spatial isolation partitions software into distinct memory spaces, which prevents separate domains from sharing the same underlying hardware. This separation is most commonly used for DRAM memory, where the OS kernel rely on hardware support like the memory management units (MMUs) or memory protection units (MPUs). The MMU translates virtual memory addresses into physical addresses in main memory and the OS can separate different processes usage entirely. It also generally provides memory protection by implementing access control rules and regulations stopping illegal usage of particular memory locations.

The latter isolation type, namely temporal isolation, must ensure that the service received from shared resources by the software in one domain cannot be affected by the software in another domain [17]. An example, which would require such isolation is any shared part of the OS between different domains. Calling shared OS code should not have any measurable timing affect for another domain calling the same OS code. The solution is often to reset the shared resource back to a known default before any partitioned item is run. This guarantees that each process runs from a clean state which is unaffected by previous use.

To achieve temporal isolation one must clean any microarchitectural state that could be affected by process usage. This is where an instruction such as `fence.t` [4] would come into play. The instruction clears any shared microarchitectural state that cannot be partitioned spatially allowing for a deterministic state for a new process to run in.

2.4 User-level mitigations

This gives way to what is known as time protection. Time protection is defined within [1] as "A collection of OS mechanisms which jointly prevent interference between security domains that would make execution speed in one domain dependent on the activities of another". An OS, which implements Time protection must make it such, that all execution within partitioned parts does not depend on each other making both covert channels and side channels infeasible. This is quite a strict version of Time Protection, and to close covert channels it must be, but quite a significant amount of timing channels can be closed at the user-level if one allows a distinction between different information types.

One such example is the FaCT DSL [3]. FaCT sought to address the challenges of writing constant-time cryptographic code, which does not depend on "secret" information. That is, instead of treating all information as possible leakage, one indicates what information must not be leaked through a timing channels, and the FaCT type system guarantees that in isolated execution this will not leak. FaCT works by making the code constant time in regards to the sensitive information, and any branching is only dependent on non-sensitive information. The idea being, that any attack can not measure the execution time of the encryption code to gather information about sensitive information. This does work to an extent, by guaranteeing constant time at the C program level, but compiler introduced and variable execution time of instructions based on inputs can still leak sensitive information. For example, the division operator is notoriously known to have its execution dependent on the numerator [18], [19]. Mitigations against these attacks would still require help from hardware, and make use of a specific instructions that guarantee a constant execution time. Further, while the execution itself might be constant time, any cache usage or shared resource beyond the CPU itself must still be partitioned as to not inadvertently leak information through the shared state. As such, even though user level mitigations can close some timing channels it is far from everything and a reset of shared resources through instructions such as the temporal fence fence[4] are still needed even if the C code itself seems "safe".

Furthermore, information that the operating system holds about a given process could also be deemed sensitive. For example, in highly secure environments the runability of a task could be deemed sensitive, and no changes on the user level side will ever close such a timing channels, as the OS scheduler will unknowingly always leak this information through its scheduling decisions. Therefore, while user level implementations do help significantly, the issue can not be fully closed without both the support of the OS and hardware.

3 Related Work

3.1 The missing OS Abstraction: seL4

The paper Time Protection: The missing OS Abstraction [1] presents a customized version of the seL4 microkernel that aims to protect against all possible timing channels in the OS.

3.1.1 Threat model

The paper’s threat model consists of a victim program containing a Trojan that uses any means necessary to leak information through timing channels. The only restriction on the Trojan is that it must run in a separate domain from the receiver/spy program.

Specifically, they differentiate between two types of channels: the **covert** channel and the **side** channel. The covert channel requires collusion between the sender and the receiver and represents the worst-case scenario, in which the sender intentionally attempts to transmit information. The side channel instead occurs when the sender inadvertently leaks information to the receiver through normal program execution. By protecting against the covert channel, it immediately follows that the side channel is also protected.

3.1.2 Channels

Two main categories of channel are referenced throughout the paper. On-core state - including L1 and L2 caches, Translation Lookaside Buffers, and other microarchitectural structures - which is unique to each core, and shared state, which includes higher-level caches that are shared between different cores. To protect against all channels, they provide the following five requirements:

- *Flush on-core state*: When time-sharing a core, all on-core microarchitectural state must be flushed on every domain switch, unless the hardware supports specific partitioning of this state.
- *Partition the OS*: Each security domain must have a private copy of OS text, stack and global data. That is, all the dynamic OS kernel memory must have the same separation as userland execution.
- *Deterministic data sharing*: Access to any remaining shared OS data must be sufficiently deterministic to avoid information leakage.

- *Flush deterministically*: State flushing must be padded to its worst case latency.
- *Partition interrupts*: When sharing a core, the OS must disable or partition any interrupts other than the preemption timer. (This is only a concern intra-core).

3.1.3 Cloning the OS

The paper introduces a method for cloning the OS. A userland program with the capability to clone gives up a subset of its own allocated memory for the clone call, and the kernel then creates a kernel image copy in the user-supplied memory. This memory is then marked as kernel data, meaning that userland cannot modify it, but any system calls will run through this kernel copy.

This mechanism leverages seL4’s security domains. The initial kernel creates a master `Kernel_Image` that represents the current kernel and includes the clone right. It hands this capability, along with the image size, to the initial user process, which then partitions the memory into security domains and clones the OS into each of these new domains.

The cloning process consists of three steps:

1. The user process retypes some Untyped memory into an uninitialized `Kernel_Image` and `Kernel_Memory` of sufficient size.
2. It allocates an address space identifier to the uninitialized `Kernel_Image`.
3. It invokes `Kernel_Clone` on the new `Kernel_Image`.

3.1.4 Cache coloring

To protect caches that are too large to be flushed, cache coloring is implemented. This ensures that separate security domains cannot access the same parts of cache, eliminating mutual influence. Cache coloring reduces the total amount of available cache space for each domain, meaning that the process creating the security domains must account for this factor so that the distribution of cache space is sufficient for all domains.

3.2 temporal_fence_t

Following the difficulties identified in [1], it was clear that hardware improvements are needed before time protection can be fully achieved. The paper [4] introduce one such hardware improvement. First, it demonstrates the presence of serious timing

channels within an in-order RISC-V core and confirms that the current ISA lacks mechanisms to close them reliably.

Following this, they present two versions of the `fence.t` instruction: a basic flush that exclusively flushes the L1 data and instruction caches, the TLBs, and the branch predictors. Through their evaluation they find that further stateful components - such as the LFSR generating a pseudo-random index sequence for the cache replacement policy and the round-robin memory arbiters of the core - still leak data through the microarchitectural state. As such, they also implement a Full Flush, which clears these components along with those covered by the basic flush. The `fence.t` instruction proceeds as follows:

- Save the program counter.
- Save locally modified (dirty) architectural state.
- Drain pending transactions.
- Clear components that are not cleared on reset.
- Assert Microreset. Clearing all flip-flops containing non-architectural state.

A full explanation of each step is provided in [4].

Given this implementation, they augmented the seL4 kernel to call the `fence.t` instruction on every context switch, finding an overhead of around 0.7%. They note that this should be seen as a pessimistic result due to the use of idle tasks, and that in a setting with high memory contention the latency may be almost entirely hidden.

3.2.1 With software support

While `fence.t` was effective and inexpensive on simple in-order cores, a follow-up paper [5] argues that out-of-order execution CPUs introduce challenges for the implementation. They address these challenges by presenting a software-supported temporal fence `fence.t.s` methodology. They show that this software-supported version is able to close all on-core timing channels without measurable hardware overhead, and with a minimal average performance impact of 1.0%.

3.3 S3K

S3K is a real-time operating system that provides a capability-based multicore partitioning kernel for embedded RISC-V systems. It provides spatial and temporal

isolation¹, time protection, and dynamic resource reconfiguration. Furthermore, S3K’s scheduler guarantees deterministic process dispatch, free from microarchitectural interference. [2]

It achieves this through three main attributes:

- A capability model.
- A time-driven, capability-based scheduler with systematic use of `fence.t`, worst-time padding, and constant-time kernel operations.
- Predictable, constant-time system calls.

3.3.1 Implementation

The implementation provides a minimal trusted computing base (TCB) with strong isolation and security guarantees. The basic execution unit in S3K is a *process*, a self-contained entity whose resources are defined by its capabilities. These processes are designed to host bare-metal real-time operating systems or applications, with the kernel acting as a bare-metal hypervisor. The number of processes is statically defined at compile time. Processes use *monitor capabilities* and capability revocation to manage other processes, reclaim resources, and repurpose existing processes for new tasks. [2]

Capabilities: Every S3K capability operation has a bounded worst-case execution time (WCET) and is free of timing side-channels. Capabilities are implemented in software and stored in a capability derivation tree (CDT), which resides in a statically allocated capability table (ctable) in memory.

Scheduling: S3K’s scheduler is a partitioned scheduler that divides time into *major frames*, which are further partitioned into *minor frames*. S3K treats minor frames as first-class capabilities, enabling dynamic reconfiguration of computing resources while maintaining process isolation. [2]

A major frame is divided into a fixed number of equal duration time slots. A single time slot represents the smallest scheduling unit. A time slice capability grants control over a range of time slots on a hardware thread (hart). The process that owns the time slice capability receives a minor frame of its own, whose length is less than or equal to the major frame. Each time slice capability has an *enable* field that specifies

¹Spatial isolation means separation of memory, whereas temporal isolation means that the failure of one component should not affect the schedulability of another.

whether the minor frame is active. If set, the owner of the time slice is scheduled during the corresponding minor frame; otherwise the minor frame represents idle time.

The kernel is in principle non-preemptable. It uses preemption points to ensure that the WCET is bounded. The kernel checks for preemption at the start of every system call and aborts the system call if preemption is required.

By the definition of domains in S3K, processes from different domains are never scheduled in parallel. Thus, during a domain switch, all running processes of the current domain are preempted, invoke the scheduler, and are subsequently delayed by the temporal fence.

3.3.2 Time protection

S3K uses the temporal fence instruction at context switches to flush the core-local microarchitectural state.

S3K is designed to reside entirely within the small scratchpad memory (SPM) backed by the core-local L1 cache. This enables the kernel’s microarchitectural footprint to be efficiently flushed and avoids reliance on larger caches.

S3K only protects a process’s in-kernel state from side-channels; it does not protect a process’s user-level state. Each process is responsible for protecting its own user-level execution from side-channels in higher level caches. This is different from the approach in [1], where cache coloring guarantees that processes in different domains never access the same parts of the cache, thereby preventing side channels even in larger caches.

All system calls are designed to be non-interfering and constant-time with respect to the invoking process’s authorized observations; that is, all control flow decisions and memory accesses depend only on data the process is permitted to observe.

3.3.3 Evaluation

They evaluate the implementation across several factors, the most significant for our work being scheduling jitter and side-channels. They first find that Inter-Process Communication (IPC) operations have the highest recorded non-preemptable cost of 1,195 CPU cycles. Together with an estimated 16,000 cycles for data cache write-back during the temporal fence, they set the CSPAD to 18,000 cycles for the `fence.t` instruction. They make use of these findings for the WCET padding throughout the codebase.

Scheduling Jitter: A measurer process, scheduled once per major frame, records the *mcycle* performance counter value upon dispatch. To simulate interference, they introduce interfering processes that execute random system calls and pollute the cache. They find that when the measurer exclusively occupies the SPM, the other processes have no impact on the measurements. However, when the measurer occupies DRAM, noticeable jitter appears. They note that in some cases this jitter originates from the small portion of code in the measurer that records the dispatch time, rather than from the scheduler itself. When more than one process is scheduled at a time, jitter also appears in the scheduler.

Side channels: They implement a Trojan process and a spy process. The Trojan signals a "0" by remaining idle and a "1" by behaving like the interfering processes from before. The spy then measures jitter, system call execution times, and cache access times to infer the Trojan's signal.

They find that when the measurer and Trojan both reside in DRAM, there is a near perfect information flow. In contrast, when both reside in the SPM, there is no measurable amount of information flow.

3.4 FreeRTOS

FreeRTOS is a class of real-time operating systems (RTOS) designed to be small enough for microcontrollers, while not limiting its application to microcontrollers alone. Out of the box, FreeRTOS provides core real-time scheduling functionality, inter-task communication, timing and synchronisation primitives, along with other optional add-on features. [20]

FreeRTOS-MPU is a memory-protected version of FreeRTOS, which at the time of writing is supported on select ARM-based microcontrollers. It enables tasks to run in either privileged or unprivileged mode and restricts access to resources such as RAM, executable code, peripherals, and memory beyond the limit of a task's stack. [21]

A third-party port of FreeRTOS-MPU to the RISC-V ISA currently exists [22]. This port forms the basis for this thesis's implementation. There are several reasons for this choice. The first and most important is familiarity: I have previously worked with the RISC-V ISA, which reduces the overhead of adopting a different ISA. Second, the related work on timing channels generally focuses on RISC-V, so future comparisons between implementations are more relevant if all are built on the same platform. Finally, the `fence.t` instruction introduced in Section 3.2 is designed with the RISC-V instruction set in mind. While the underlying idea could

be implemented on other platforms, work on hardware support for those platforms is not as far along. Therefore, assuming this implementation will eventually run on real hardware, RISC-V seems the most reasonable choice.

4 Scheduling

4.1 The FreeRTOS scheduler

FreeRTOS provides different scheduling algorithms based on configurations and number of cores available on the system. By default on a single core FreeRTOS makes use of a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks. [23]

As such, FreeRTOS always attempts to schedule the highest priority task and possibly starves lower priority tasks. Items of the same priority run until a tick interrupt at which point the time slice is up. This type of scheduling inherently includes timing side-channels as the higher priority tasks inhibit lower priority tasks from running. [24], [25] This could be used as a covert channel, where the higher priority task could signal either a 0 or 1 by simply going to sleep and the lower priority task measures its own execution time.

Therefore, for FreeRTOS to provide timing protection the scheduler must see modifications.

4.2 Closing Scheduling Based Timing Channels

There are a few approaches to closing the different side channels each with their own pros and cons. A general attribute among all of them is, that starvation must not be possible, and as such the current approach to priority based scheduling is infeasible.

Removing time: One approach to solve a lot of the timing channels would be to disallow tasks to read the current time. This in theory would remove the scheduling based timing channel, but this solution would most likely attributed quite a significant decrease in the possibilities of tasks, as any action, which could leak time information to the tasks would need to be prohibited. As such, this approach was not considered further.

Protection within priority: If the goal is to keep a sense of priority based scheduling, then it should be possible to guarantee timing protection for tasks of same priority or tasks of different priorities. Say for example, we for each priority

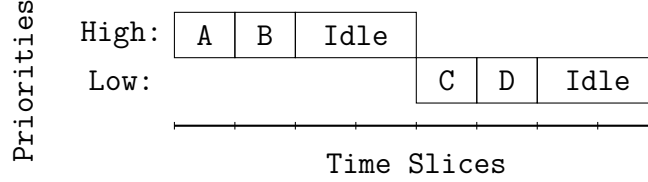
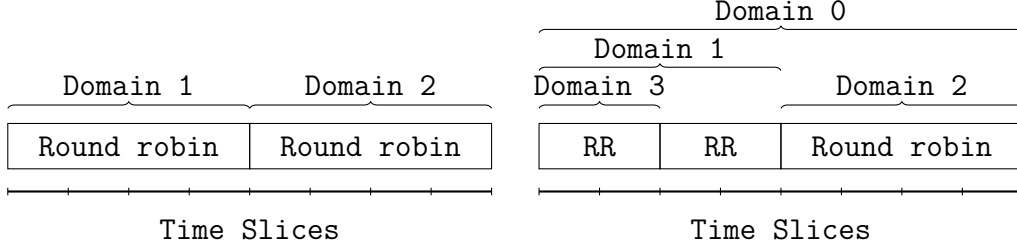


Figure 1: Time protection within same priority

have a known array which constitutes the time slices for the given priority. When the scheduler attempts to schedule a task, then it prioritizes the higher priority array if a task is ready within, and uses time slice based scheduling within the priority. An example of this can be seen visualized in Figure 1, where tasks A and B are within the same priority. Here, if we assume that A and B are runnable, then they would be scheduled before the lower priority C and D. This would allow C and D to gather information about the runnability of the entire High priority noting that either A or B was runnable if C or D are delayed. However, within the same domain tasks A and B can not gather information about each other. Say for example A gets to run, then it does not know whether task B is runnable or not, as we are always running the entire time slice given defined by the aforementioned array.

This approach would by no means close all timing channels, as mentioned cross priorities could still leak timing information. Other constraints with this approach is, that we are limiting the number of possible tasks for given priorities. In the previous example 4 time slices were provided within the High and Low priorities, and we would not be able to fill more than these 4 time slices as we would then change the total time slice when running a specific priority and thus a task within the given priority could gather information about the total number of time slices. Another issue would be, that we can't simply give up CPU time. When a given tasks wishes to sleep for a short duration, we must replace the time slice with idle time, which significantly reduces overall CPU utilization. Once a task is completely finished it is possible to then take back the time slice, but the new task that would be scheduled in this slot would then gather the information that the previous task finished its execution.

Domain scheduling: Another approach is the one seen in [1], which is the domain level scheduler. The general idea is quite similar to the approach seen before, but the



(a) Static domain scheduler

(b) Dynamic domain scheduler

Figure 2: Domain scheduling

idea of priorities is removed entirely and instead we focus on domains. Each domain can have multiple tasks within, where we do not care about timing channels and only between different domains do we care about timing protection. In Figure 2a an illustration of this scheduler can be seen. The idea is, that at compile time, we know the different domains and attribute to each some amount of time slices (in the example that would be 4 for both domain 1 and 2). The domain scheduler then takes care of scheduling the domain, and guarantees timing protection only between the different domains. This approach is still inhibited by the need to know the total number of time slices at compile time, and further it also needs the number of time slices for each domain at compile time. However, where this benefits over the previous approach is, that within each domain there is a lot more freedom to do as pleased. For example, one could imagine having a regular round robin scheduler within a domain, which then allows more CPU utilization within this specific domain and less unused resources.

An extension of this approach would be the one seen in Figure 2b, which closely follows the capability based scheduler in [2]. The idea is, that within a specific domain, we can give up some of the attributed time to create a new subdomain. This would mean that there is timing protection between subdomain 1 and the rest of domain 1². In theory it should be possible to give up these time slices back to the domain level scheduler, which would then treat this as a new domain. Whether or not some timing channels persists after creation remains to be seen.

This approach seems by far the most flexible and most robust in terms of real

²Other than creation and finishing assuming a domain is allowed to give back the time slices upon finishing, which would leak timing information

world applications. As such, this thesis attempts to implement this domain scheduler within the FreeRTOS kernel.

It is important to note, that this approach does not remove all timing channels. Each domain uses a round robin scheduler, which as mentioned previously is susceptible to timing channels. The idea is instead, that we allow such channels in tasks where it is not relevant for protection and throughput is valued higher. Instead, when we explicitly want a safe execution environment we can create a domain for this, which is then protected after initialization.

4.3 Implementation

4.3.1 Extending the api

To implement the domain level scheduler we have first incorporate the idea into the existing FreeRTOS framework. With FreeRTOS MPU, we are able to create unprivileged tasks which have restricted access to syscalls and provide them with a memory space that is partitioned. For this, the general FreeRTOS api is extended with a multitude of new functions most importantly for this project is the `xTaskCreateRestricted`.^[21] The idea behind the domain scheduling fits with the purpose of memory protection, and as such it seems the obvious choice for modification to extend it with the domain functionality. The signature of the aforementioned function is provided in the listing below:

```
1 typedef struct xMEMORY_REGION
2 {
3     void * pvBaseAddress;
4     uint32_t ulLengthInBytes;
5     uint32_t ulParameters;
6 } MemoryRegion_t;
7
8 typedef struct xTASK_PARAMETERS
9 {
10     TaskFunction_t pvTaskCode;
11     const char * pcName;
12     configSTACK_DEPTH_TYPE usStackDepth;
13     void * pvParameters;
14     UBaseType_t uxPriority;
15     StackType_t * puxStackBuffer;
16     MemoryRegion_t xRegions[ portNUM_CONFIGURABLE_REGIONS ];
17     #if (
18         ( portUSING_MPU_WRAPPERS == 1 ) &&
19         ( configSUPPORT_STATIC_ALLOCATION == 1 )
```

```

20     )
21     StaticTask_t * const pxTaskBuffer;
22     #endif
23 } TaskParameters_t;
24
25 BaseType_t xTaskCreateRestricted(
26     const TaskParameters_t * const pxTaskDefinition,
27     TaskHandle_t * pxCreatedTask
28 ) PRIVILEGED_FUNCTION;

```

Listing 1: xTaskCreateRestricted function signature

That is, we can create a task which has access to a number of memory regions defined by the region start address (pvBaseAddress), the region size in bytes (ulLengthInBytes) and the access permissions for the region (ulParameters). The function parameter uxPriority is also of importance. By default this function creates tasks running in user mode, which restricts access to specific system calls and any task created running in privileged mode is able to access the entire RAM. By default, tasks running in user mode are able to create other tasks running in user mode, and this is the desired behaviour for domain scheduling.

The idea is to extend this function signature as follows:

```

1  ...
2
3  typedef struct xDOMAIN_PARAMETERS
4  {
5      uint32_t ulSliceIndex;
6      uint32_t ulSliceLength;
7  } DomainParameters_t;
8
9  typedef struct xTASK_PARAMETERS
10 {
11     ...
12     #if ( configENABLED_DOMAINS == 1 )
13         DomainParameters_t * xDomainParameters;
14     #endif
15 } TaskParameters_t;
16
17 BaseType_t xTaskCreateRestricted(
18     const TaskParameters_t * const pxTaskDefinition,
19     TaskHandle_t * pxCreatedTask
20 ) PRIVILEGED_FUNCTION;

```

Listing 2: Extended parameters and signature

A task with the capability to create a new restricted task would then have to provide the index in the global time slice array, along with the length of the new domain that will be created. If no `xDomainParameters` is passed to the function (i.e. the pointer is NULL) this will be treated as creating a task within the same domain, and will be handled by the per domain priority based scheduler.

With this, no more of the external api has to be changed.

4.3.2 New configuration flags

To enable the functionality of domain level scheduling, a few new compile flags need to be introduced.

configENABLE_DOMAINS A binary flag to indicate whether or not to use the domain level scheduler alongside the default priority based scheduler.

configNUM_TIME_SLICES The total number of time slices for the domain scheduler to take into account. As mentioned before this flag also constitutes the maximum number of possible domains, such that any domain scheduling decisions can be done in constant time.

configNUM_TICKS_PER_SLICE The number of timer interrupt ticks to constitute a single time slice. Say that the ticks is defined as `#define configTICK_RATE_HZ ((TickType_t) 1000)` with `#define configNUM_TICKS_PER_SLICE (5)`, this would mean that a single time slice is equal to $TIME_SLICE = \frac{1s}{1000} \cdot 5 = 5ms$.

4.3.3 Domain switching

To keep with the fundamentals of FreeRTOS the idea is to keep the functionality of the priority based scheduler within FreeRTOS but internally in a domain. That is, within a specific domain the same priority based scheduler is available. This in turn means that if no new domains are created, then the functionality of the scheduler should be nearly unchanged³.

FreeRTOS makes use of a single static file `tasks.c` for the development of the priority based scheduler. In this thesis I am only focused on the single core implementation and while work on the multicore version would be interesting, that is left for future work.

³There will be some overhead with the implementation of course.

In the MPU version of FreeRTOS when a machine timer interrupt occurs it is handled in the following manner:

```

1 freertos_risc_v_mtimer_interrupt_handler:
2     portcontextSAVE_INTERRUPT_CONTEXT
3     portUPDATE_MTIMER_COMPARE_REGISTER
4     call xTaskIncrementTick
5     beqz a0, exit_without_context_switch
6     call vTaskSwitchContext

```

Listing 3: FreeRTOS mtimer interrupt handler

It increments a global tick, and switches the task context. However, FreeRTOS only increments the global `xTickCount` if the priority based scheduler is running and not suspended for critical handling. This means, we have to introduce our own proxy for Ticks, as the per domain `xTickCount` would leak information about other scheduler states and possibly delay the domain switching. For our purposes we can hook into the pre-existing `xTaskIncrementTick` function, and check for a domain switch.

```

1  /* Static variable to contain the current Domain Tick */
2
3  PRIVILEGED_DATA static size_t pxCurrentDomainIndex = 0;
4  PRIVILEGED_DATA static volatile TickType_t xEffectiveTick = ( TickType_t )
5      ↪ configINITIAL_TICK_COUNT;
6  PRIVILEGED_DATA static volatile TickType_t xDomainTick = ( TickType_t ) 0;
7
8  BaseType_t xTaskIncrementTick( void ) {
9      ...
10     const TickType_t xConstTickCount = xEffectiveTick + ( TickType_t ) 1;
11
12     /* Increment the effective tick. This is different than xTickCount,
13     * which does not increment if the scheduler is in a critical section.
14     * The Domain scheduler does not care about critical section */
15     xEffectiveTick = xConstTickCount;
16
17     /* Update the Domain tick in regards to the effective tick */
18     xDomainTick = (xEffectiveTick / configNUM_TICKS_PER_SLICE) %
19         ↪ configNUM_TIME_SLICES;
20
21     /* Time slot expired or wrap-around */
22     if( xDomainTick >= pxCurrentDomainIndex +
23         ↪ xDomains[pxCurrentDomainIndex].uxLength
24         || xDomainTick < pxCurrentDomainIndex )

```

```

22 {
23     /* Save current task to logical domain info */
24     xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB =
    ↪ pxCurrentTCB;
25
26     xSwitchRequired = pdTRUE;
27
28     /* Flush on domain switch */
29     temporal_fence_t();
30
31     /* Contiguous partition: next block starts exactly here */
32     pxCurrentDomainIndex += xDomains[pxCurrentDomainIndex].uxLength;
33     if( pxCurrentDomainIndex >= configNUM_TIME_SLICES )
34     {
35         pxCurrentDomainIndex = 0;
36     }
37
38     /* Restore previous task from logical domain info */
39     TCB_t * previous_tcb = \
40         xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB;
41     /* First time, use idle task */
42     if (previous_tcb == NULL) {
43         pxCurrentTCB = xIdleTaskHandles[ 0 ];
44     } else {
45         pxCurrentTCB = previous_tcb;
46     }
47 }
48 ...
49 }

```

Listing 4: Custom hook for xTaskIncrementTick

As such, whenever there is a machine timer tick, we calculate in constant time the time slice (`xDomainTick`) and check if we should do a domain switch. If we find that we should schedule a new domain, then we switch the data to the new domain and perform a temporal flush to clear the microarchitectural state. The flush happens in such a manner, that any data accessed in regards to the new domain is non-interfering with the previous domain. Once we have decided that a switch should occur, only data that is within the incoming domain is retrieved following the flush, this means that the new domains execution will not be based on the microarchitectural state of the previous domain.

Note this comes with some constraints. Most notably we are defining a time slice in relation to the machine timer interrupt timing, which is configurable in FreeRTOS through the `configTICK_RATE_HZ`. To make this more flexible, the `configNUM_`-

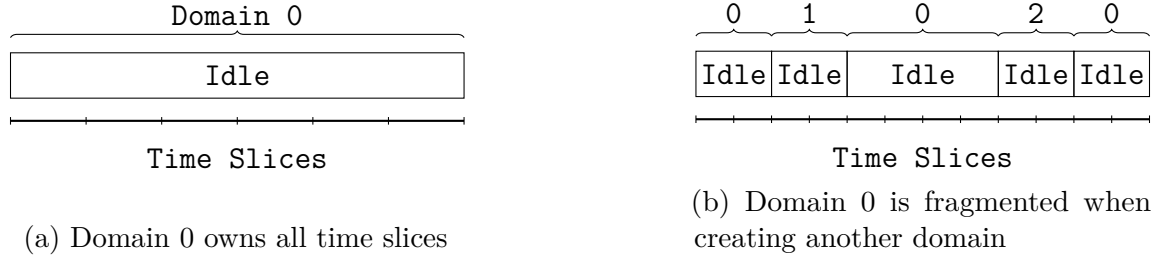


Figure 3: Example timelines when creating domains

TICKS_PER_SLICE as to not limit the domain level scheduler with respect to the inter domain priority based scheduler.

4.3.4 Constant time domain creation

To keep track of the different domains new global constants are defined:

```

1 typedef struct DomainBlock {
2     UBaseType_t uxDomainID; /* This time slices domain ID */
3     size_t uxLength; /* Length of this domain */
4     size_t uxNext;
5 } DomainBlock_t;
6
7 /* Timeline over configNUM_TIME_SLICES, indexed by the start of a block */
8 PRIVILEGED_DATA static DomainBlock_t xDomains[configNUM_TIME_SLICES] = {
9     [0] = {.uxDomainID = 0, .uxLength = configNUM_TIME_SLICES, .uxNext =
10         ↪ SIZE_MAX}
11 };

```

Listing 5: Domain timeline

For now the main data structure introduced is an array, namely `xDomains`, storing the timeline. This array is indexable by the start time of the current group of time slices and contains information about the current group of domain slices, such as the length. Two example states of this array can be seen visualized in Figure 3. For the case where Domain 0 owns all time slices `xDomains` would only contain a single valid index, namely the 0 index, which stores that it is domain 0, that owns the group of time slices and the group has a length of 6.

This way, our previous domain switching can be directly indexed once we find that a new domain switch needs to happen providing constant time retrieval of the next domain.

At first glance domain creation also seems simple. In Figure 3a there are three main cases for domain creation. Either we create domains by giving up time slices at the start or the end of our own time slice, which would simply create two separate domains with no "split" domain. If instead a domain is created in the middle of the current domain, then the current domain is split and might be scheduled several times during a single round trip of the time slices. Such an example can be seen in Figure 3b, where Domain 0 runs at separate times of execution. This poses an issue when wanting to create yet another domain. As mentioned in Section 4.3.1, a user of the OS would provide the time slice for which to start a new domain and the length of the domain. Given that not all indices throughout the domain are defined, we would need to scan the array from left to right to figure out, if the current domain truly does "own" the given time slices the user is attempting to access. If a user wants to create a new domain from Domain 0s time slice 4-6 (0 indexed), then the naive method would loop from the start of the array until reaching the group of time slices, which contains these time slices, namely indices 4-8. This execution is not constant time with respect to what domain 0 should know as the execution time of the loop depends on how many other domains are created before no matter which domain owns those time slices.

To solve this issue the `uxNext` field was introduced, which contains the index of the next group of time slices owned by the current domain. This field can then be used for the looping, when finding the encompassing group of time slices giving us execution that is only dependent on information available to the current domain. In the case that there is no next domain the `SIZE_MAX` is used.

With this in place, it allows finding the encompassing domain as follows:

```

1 static BaseType_t prvFindContainingBlock(
2     size_t uxFirstBlock,
3     size_t start,
4     size_t length,
5     size_t * puxPrevBlock,
6     size_t * puxBlock)
7 {
8     size_t prev = SIZE_MAX;
9
10    for (size_t current = uxFirstBlock; current != SIZE_MAX; current =
11        ↪ xDomains[current].uxNext)
12    {
13        size_t blockEnd = current + xDomains[current].uxLength;
14
15        if (start >= current && (start + length) <= blockEnd)
16        {

```

```

16         *puxPrevBlock = prev;
17         *puxBlock = current;
18         return pdTRUE;
19     }
20
21     prev = current;
22 }
23
24 return pdFAIL;
25 }

```

Once we have found the encompassing group of time slices we have to create the new domain, which is done via index arithmetic as seen below:

```

1  ...
2  /* Right fragment */
3  size_t rightStart = SIZE_MAX;
4  if (start + length < blockEnd)
5  {
6      rightStart = start + length;
7      xDomains[rightStart].uxDomainID = uxCurrentDomainID;
8      xDomains[rightStart].uxLength = blockEnd - rightStart;
9      xDomains[rightStart].uxNext = xDomains[blockStart].uxNext;
10 }
11
12 /* Left fragment: shrink existing block */
13 if (start > blockStart)
14 {
15     xDomains[blockStart].uxLength = start - blockStart;
16     if (rightStart != SIZE_MAX)
17     {
18         xDomains[blockStart].uxNext = rightStart;
19     }
20 }
21 else /* No left fragment: remove blockStart from parent's list */
22 {
23     size_t spliceNext = (rightStart != SIZE_MAX) ? rightStart :
24     ↪ xDomains[blockStart].uxNext;
25     if (prev == SIZE_MAX)
26     {
27         xDomainInfo[uxCurrentDomainID].uxFirstBlock = spliceNext;
28     }
29     else
30     {
31         xDomains[prev].uxNext = spliceNext;

```



```

31     }
32 }
33 ...

```

We first check, if we would create a new right fragment of the current domain. It would exist if we are attempting to create a new domain, which starts before the current domain. If such a fragment exists, then we update the index in `xDomains` to track it.

Next, we can check if a left fragment is created, and simply shrink the encompassing time slice group and updating the `uxNext` field indicating that we have been split and the next block owned by the current domain is that of the new right fragment.

If there is no left created, then we are creating a new domain at the start of the encompassing block, so we remove the encompassing block start from the linked list, and update it to point to the right fragment if it was created or the next block. In the case that there is no next block, then it was previously set to `SIZE_MAX` indicating again, that there is no next block.

At last, we have to update the `xDomains` to include the newly created domain, which is simply done as follows:

```

1  ...
2  UBaseType_t uxChildDomainID = uxNextDomainID++;
3  xDomains[start].uxDomainID = uxChildDomainID;
4  xDomains[start].uxLength = length;
5  xDomains[start].uxNext = SIZE_MAX;
6
7  xDomainInfo[uxChildDomainID].pxPreviousTCB = NULL;
8  xDomainInfo[uxChildDomainID].uxFirstBlock = start;
9  ...

```

The global `xDomainInfo` is the array used for scheduling the TCB within each domain, and is indexable by the domain id rather than the start of the time slice block.

4.3.5 Per domain scheduling

For the domain level scheduler to work with inter domain priority based schedulers, we need to add another dimension to all of the items. That is, all variables that previously were a constant would have to be an array with length equal to the number of time slices and same extension for previous arrays. All accesses to these constants/arrays would have to follow a similar transformation, where the currently

scheduled domain would choose the correct values for the priority based scheduler to function correctly. A few examples of these transformations are shown below:

```

1  /* Non-domain scheduler implementation */
2  PRIVILEGED_DATA static volatile BaseType_t xSchedulerRunning = pdFALSE;
3  PRIVILEGED_DATA static volatile TickType_t xPendedTicks = ( TickType_t ) 0U;
4
5  /* Domain scheduler equivalent */
6  PRIVILEGED_DATA static volatile BaseType_t
   ↪ xSchedulerRunning[configNUM_TIME_SLICES] = { pdFALSE };
7  PRIVILEGED_DATA static volatile TickType_t
   ↪ xPendedTicks[configNUM_TIME_SLICES] = {( TickType_t ) 0U};

```

Listing 6: Transforming the priority based scheduler

Note, that while the first element is initialized here, the initialization of the full items are moved to a separate function, as all of the schedulers would have to be initialized in the same fashion as the default scheduler.

With this, the general foundation of the domain level scheduler is functional, and the standard `mpu_blinky.c` example works given all tasks run within the same domain.

4.3.6 Cross domain communication

As it stands, the FreeRTOS api has next to no information about the different domains, meaning that items such as the Queue used for message transferring between domains is not available. Generally this is a good thing, as by default there is domain isolation, as any internal handling within `tasks.c` makes use of the current domain. This restriction is quite strict, and one might assume that cross-domain communication should be possible. In FreeRTOS communication happens through Queues. The general workflow for Queue communication in the MPU version of FreeRTOS is as follows:

- Create Queue kernel object in a priority task.
- Grant access to the Queue object to any user level tasks, that should communicate through the Queue.
- One task sends messages through `xQueueSend`.
- Another task receives messages through `xQueueReceive`.

For the cross domain communication the workflow should be the same, but it is the internal handling that has to change. Currently the internal handling of the `xQueueSend` is:

- Check there is room in the queue for the message.
- Check if there is a task waiting to receive on the Queue.
 - If yes, move the task to the ready list.
 - If no, do nothing.

This is generally also fine, except that when moving a task to the ready list, the task might be outside the domain that is currently running. When the Queue moves a task to the ready list, it does so through the function `xTaskRemoveFromEventList`. This function takes a linked list of tasks, namely the tasks waiting on the queue, removes the first task from the list and moves the given task into the ready list if the scheduler is suspended and otherwise moves it to the `xPendingReadyList` to wait for the next scheduler suspension before moving into the true ready list. If the new task was found to have a higher priority, then notify that there should be a context switch.

This is where we have to make changes to allow for cross-domain communication. First, each task needs some way of tracking the domain it resides within, which is easily done by extending the `TCB_t` struct with an extra field:

```
1 typedef struct tskTaskControlBlock
2 {
3     ...
4     #if ( configENABLE_DOMAINS == 1 )
5         UBaseType_t uxDomainID;
6     #endif
7 } tskTCB;
8
9 typedef tskTCB TCB_t;
```

The Queue can now reference this `DomainID` when modifying the priority-schedulers and as such adds the tasks to the correct arrays.

Given this communication, we can now create the `mpu_blinky.c` example with different domains communicating. It is important to note here, that there of course is introduced some timing leakage between these two domains, as they can time the response time of different messages. Here it is up to the individual domains to respond in a deterministic manner as to not leak any secret information.

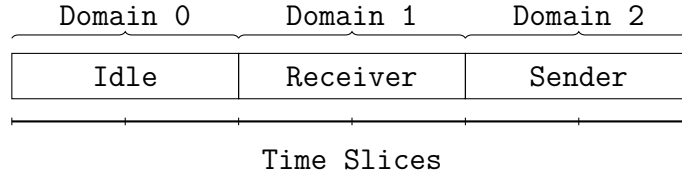


Figure 4: Timeline of the Blinky Domain Scheduling

5 Evaluation

5.1 The blinky example

The RISC-V mpu port includes a standard `mpu_blinky.c` example. The example program creates two tasks, a sender and a receiver. The sender adds a message to a `Queue` at a set interval, and the receiver reads from the `Queue` whenever there is a message available. After a set amount of message passing, the sender reads from a restricted memory address causing a pmp fault that gets handled by a custom exception handler. For cross-domain communication, a similar approach can be used to check that the tasks are still capable of passing messages to each other. The domain level scheduler is setup such, that the main function is called with elevated privilege and it is capable of creating queues and new tasks. The tasks creation is modified to make use of the new api, such that we create two new tasks, each in a separate domain. The scheduling timeline of this modified example is provided in Figure 4.

That is, when the initial program is started and we are within domain 0, then domain 0 has given up 2 time slices for the Sender (domain 2), and 2 time slices for the Receiver domain (Domain 1). As it itself finishes execution, the rest of its time slices will simply be used by the idle task. Each task is capable of reading the current domain tick, and the demo simply prints this, when execution starts. Running the demo, we get the following snippet of output:

```

1 Sending to task, DomainTick: 4
2 Message received from task, DomainTick: 2
3 Sending to task, DomainTick: 4
4 Message received from task, DomainTick: 2
5 Sending to task, DomainTick: 4

```

```

6 Message received from task, DomainTick: 2
7 Sending to task, DomainTick: 4
8 Message received from task, DomainTick: 2
9 Sending to task, DomainTick: 4
10 prvQueueSendTask: trying to access privileged data
11
12 === PMP ACCESS FAULT ===
13 mcause = 0x00000005
14 mepc   = 0x80003730
15 mtval  = 0x80180001
16 mstatus = 0x00000080
17 type   = load
18 task   = Tx
19 system halted.

```

The output does show some interesting properties with regards to the domain scheduler. First, although the receive task is scheduled before the sender task, given that there is no message to read in the Queue, it simply sleeps for its very first execution scheduling the Idle task. Next, we see that the sending happens always at the specified DomainTick 4, and that the Receiving also happens within the expected domain tick 2. Another interesting note is, that in the very last iteration before the access fault, the receiver tasks is never able to receive the final message, as the sender continues uninterrupted before causing the fault.

5.2 Constant time

While the previous example shows, that the Domain Tick seems constant for both⁴, it does not show that each domain runs for a constant amount of time before interruptions. To truly show that this runs in constant time, we are limited with the tools available. QEMU does not provide cycle-accurate emulation, and by default makes use of the host system clock for interrupts, meaning that the demo's execution is dependent on the rest of the host system. This issue is something that will be discussed later in Section 5.3. QEMU does however provide a flag known as `-icount`, which provides deterministic execution from the guest point of view. Instead of making use of the host interrupt, QEMU instead makes use of instruction counting to determine elapsed time [26]. While this by no means is cycle accurate and does not take into account any microarchitectural state, it can be used to approximate the time each domain is allowed to run before a scheduling occurs.

In RISC-V the `rdcycle` instruction reads XLEN bits of the `cycle` counter, which

⁴If it varies then the likely culprit is the priority based scheduler within each domain.

holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past. Given FreeRTOS runs with XLEN 32-bit, we have to also make use of the `rdcycleh` instruction, to read the upper 32 bits of the cycle counter, to get the true cycle count. This means, that we can find a round trip time of the domain switch in the following fashion:

```

1 static uint64_t ullPrevMtime = 0;
2
3 uint64_t ullMtime;
4 uint64_t ullMtimeHigh;
5
6 asm volatile ("rdtime %0" : "=r"(ullMtime));
7 asm volatile ("rdtimeh %0" : "=r"(ullMtimeHigh));
8 ullMtime = ((ullMtimeHigh << 32) | ullMtime);

```

By saving the previous cycle count from when we last reached this and calculating the round trip time, we can make the demo print this round trip time for each domain, giving us output like the following:

```

...
Sending to task, DomainTick: 4
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Message received from task, DomainTick: 2
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50001
Round Trip: domain=1 delta=49999
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Sending to task, DomainTick: 4
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000

```

```
Message received from task, DomainTick: 2
...
```

Which shows that each domain runs in the same number of cycles with a variance of 1. This variance seems negligible, and could most likely be attributed in some small discrepancy in how QEMU handles the machine timer interrupt.

5.3 QEMU cycle accuracy

As mentioned previously, the QEMU virtual machine does not provide cycle accurate emulation. The closest provided is the instruction based time counting as described in [26]. This means that any timing channel that arises from the microarchitectural shared state can not be emulated or diagnosed on the QEMU platform. What can be empirically evaluated on the platform is a constant time execution path as shown with the previous domain level scheduler. In an attempt to gain more true cycle estimations a QEMU TCG plugin was developed for emulating the cache behaviour of the program during execution. QEMU provides a cache modelling plugin example [27].

5.4 Limitations

FreeRTOS was not created with the MPU version in mind. This means that development on the MPU version of FreeRTOS is a quite recent endeavour and most likely still subject to change. One clear issue with the aforementioned domain level scheduler is the inter-domain scheduler itself. FreeRTOS provides two different modes for tasks, namely the Privileged Mode and the User mode. When a task is in user mode, it is unable to create any other tasks, as this changes kernel memory and disables interrupts for the duration of the call. This poses an issue when wanting to implement strict boundaries between separate domains. When creating a new domain, if this is initialized with only a User mode task, then that domain will see no other tasks as the task itself can create new tasks.

The more subtle problem is, that Privileged Mode takes precedence over the domain separation. If we have two domains 0 and 1, each with a single Privilege mode task within, then these two tasks can always communicate with each other, as there are no restrictions on what a Privilege task can access. The domain separation is thus only for User mode tasks, and by no means does it extent to the Privileged mode tasks within FreeRTOS. Further, the separation itself requires that the privileged mode tasks within each domain explicitly run within the allocated time slices as they can disable interrupts via the FreeRTOS api. One such example is for task creation.

When a Privileged mode task wishes to create another task, FreeRTOS enters a critical section, which for the RISC-V version disables all interrupts including the machine hardware interrupt. If the task does not do this carefully, it could delay the domain level scheduler, which would create a possible timing channel between this privileged task and a user level task in another domain. This means that the only cross-domain timing protection enabled via this implementation is between two User mode tasks running in separate domains.

5.5 The FreeRTOS kernel is vast

One requirement for time protection specified within [1] is that the OS must be partitioned. This includes all system calls, which must be constant time in regards to available information within each domain. Given that the domain scheduler implemented creates a copy of the priority based scheduler for each domain, it can be assumed that a large portion of these system calls indirectly become constant time in this aspect. Further, as the User mode tasks are unable to disable any interrupts this would indicate that the shared front that remains to be partitioned is anything regarding the domain level scheduler itself. Given that these do make use of constant time, and domain creation is restricted to Privileged mode, then it would seem that this requirements might be met to some extent. However, to truly show this partitioned OS one would need a full audit of all system calls available for User mode tasks, and make sure that they are constant time in respect to the domain available information. As this is something beyond the scope of this thesis, the implementation can provide no guarantees that it is truly implementing time protection.

6 Discussion

6.1 Partitioned hardware

6.2 External interrupts

6.3 Out of order execution

6.4 Call for safe hardware interface

7 Conclusion

References

- [1] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, “Time Protection: The Missing OS Abstraction,” in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys ’19, New York, NY, USA: Association for Computing Machinery, Mar. 25, 2019, pp. 1–17, ISBN: 978-1-4503-6281-8. DOI: 10.1145/3302424.3303976. [Online]. Available: <https://dl.acm.org/doi/10.1145/3302424.3303976> (visited on 02/03/2026).
- [2] H. Karlsson and R. Guanciale, “Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning,” in *2025 IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2025, pp. 68–81. DOI: 10.1109/RTSS66672.2025.00015. [Online]. Available: <https://ieeexplore.ieee.org/document/11315078> (visited on 02/04/2026).
- [3] S. Cauligi, G. Soeller, B. Johannesmeyer, *et al.*, “FaCT: A DSL for timing-sensitive computation,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2019, New York, NY, USA: Association for Computing Machinery, Jun. 8, 2019, pp. 174–189, ISBN: 978-1-4503-6712-7. DOI: 10.1145/3314221.3314605. [Online]. Available: <https://dl.acm.org/doi/10.1145/3314221.3314605> (visited on 02/03/2026).
- [4] N. Wistoff, M. Schneider, F. K. Gürkaynak, G. Heiser, and L. Benini, “Systematic prevention of on-core timing channels by full temporal partitioning,” *IEEE Transactions on Computers*, vol. 72, no. 5, pp. 1420–1430, May 2023, ISSN: 2326-3814. DOI: 10.1109/tc.2022.3212636. [Online]. Available: <http://dx.doi.org/10.1109/TC.2022.3212636>.

- [5] N. Wistoff, G. Heiser, and L. Benini, “Fence.t.s: Closing Timing Channels in High-Performance Out-of-Order Cores Through ISA-Supported Temporal Partitioning,” in *Applications in Electronics Pervading Industry, Environment and Society*, M. Ruoch, F. Bellotti, R. Berta, M. Martina, and P. Motto Ros, Eds., Cham: Springer Nature Switzerland, 2025, pp. 269–276, ISBN: 978-3-031-84100-2. DOI: 10.1007/978-3-031-84100-2_32.
- [6] J.-F. Dhem, F. Koeune, P.-A. Leroux, P. Mestré, J.-J. Quisquater, and J.-L. Willems, “A Practical Implementation of the Timing Attack,” in *Smart Card Research and Applications*, J.-J. Quisquater and B. Schneier, Eds., red. by G. Goos, J. Hartmanis, and J. Van Leeuwen, vol. 1820, Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 167–182, ISBN: 978-3-540-67923-3 978-3-540-44534-0. DOI: 10.1007/10721064_15. [Online]. Available: http://link.springer.com/10.1007/10721064_15 (visited on 02/17/2026).
- [7] O. Aciğmez, W. Schindler, and Ç. K. Koç, “Improving Brumley and Boneh timing attack on unprotected SSL implementations,” in *Proceedings of the 12th ACM Conference on Computer and Communications Security*, ser. CCS ’05, New York, NY, USA: Association for Computing Machinery, Nov. 7, 2005, pp. 139–146, ISBN: 978-1-59593-226-6. DOI: 10.1145/1102120.1102140. [Online]. Available: <https://dl.acm.org/doi/10.1145/1102120.1102140> (visited on 02/17/2026).
- [8] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, “Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds,” in *Proceedings of the 16th ACM Conference on Computer and Communications Security*, Chicago Illinois USA: ACM, Nov. 9, 2009, pp. 199–212, ISBN: 978-1-60558-894-0. DOI: 10.1145/1653662.1653687. [Online]. Available: <https://dl.acm.org/doi/10.1145/1653662.1653687> (visited on 02/04/2026).
- [9] M. Lipp, M. Schwarz, D. Gruss, *et al.*, “Meltdown: Reading kernel memory from user space,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.
- [10] J. Corbet, “KAISER: Hiding the kernel from user space,” *LWN.net*, Nov. 15, 2017. [Online]. Available: <https://lwn.net/Articles/738975/> (visited on 02/17/2026).
- [11] P. Kocher, J. Horn, A. Fogh, *et al.*, “Spectre attacks: Exploiting speculative execution,” in *40th IEEE Symposium on Security and Privacy (S&P’19)*, 2019.

- [12] T. Murray, D. Matichuk, M. Brassil, *et al.*, “seL4: From General Purpose to a Proof of Information Flow Enforcement,” in *2013 IEEE Symposium on Security and Privacy*, Berkeley, CA: IEEE, May 2013, pp. 415–429, ISBN: 978-0-7695-4977-4 978-1-4673-6166-8. DOI: 10.1109/SP.2013.35. [Online]. Available: <http://ieeexplore.ieee.org/document/6547124/> (visited on 02/17/2026).
- [13] S. Buckley, R. Sison, N. Wistoff, *et al.* “Proving the Absence of Microarchitectural Timing Channels.” version 1. arXiv: 2310.17046 [cs]. (Oct. 25, 2023), [Online]. Available: <http://arxiv.org/abs/2310.17046> (visited on 02/10/2026), pre-published.
- [14] M. Schneider, D. Lain, I. Puddu, N. Dutly, and S. Capkun, “Breaking Bad: How Compilers Break Constant-Time Implementations,” in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, Aug. 25, 2025, pp. 1690–1706. DOI: 10.1145/3708821.3733909. arXiv: 2410.13489 [cs]. [Online]. Available: <http://arxiv.org/abs/2410.13489> (visited on 02/17/2026).
- [15] D. A. Osvik, A. Shamir, and E. Tromer, “Cache Attacks and Countermeasures: The Case of AES,” in *Topics in Cryptology – CT-RSA 2006*, D. Pointcheval, Ed., red. by D. Hutchison, T. Kanade, J. Kittler, *et al.*, vol. 3860, Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 1–20, ISBN: 978-3-540-31033-4 978-3-540-32648-9. DOI: 10.1007/11605805_1. [Online]. Available: http://link.springer.com/10.1007/11605805_1 (visited on 02/10/2026).
- [16] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, “Last-Level Cache Side-Channel Attacks are Practical,” in *2015 IEEE Symposium on Security and Privacy*, May 2015, pp. 605–622. DOI: 10.1109/SP.2015.43. [Online]. Available: <https://ieeexplore.ieee.org/document/7163050> (visited on 02/04/2026).
- [17] J. Rushby, “Avionics Architectures: Mechanisms, and Assurance,” 1999.
- [18] R. L. Schröder, S. Gast, and Q. Guo, “Divide and Surrender: Exploiting Variable Division Instruction Timing in HQC Key Recovery Attacks,”
- [19] Y. Jin, P. Qiu, C. Wang, Y. Yang, D. Wang, and G. Qu. “Timing the Transient Execution: A New Side-Channel Attack on Intel CPUs.” arXiv: 2304.10877 [cs]. (Apr. 21, 2023), [Online]. Available: <http://arxiv.org/abs/2304.10877> (visited on 04/29/2026), pre-published.
- [20] “What is FreeRTOS? - FreeRTOS™.” (Apr. 2026), [Online]. Available: <https://freertos.org/Why-FreeRTOS/What-is-FreeRTOS> (visited on 04/07/2026).

- [21] “Memory Protection Unit (MPU) Support - FreeRTOS™.” (Apr. 2026), [Online]. Available: <https://freertos.org/Security/04-FreeRTOS-MPU-memory-protection-unit> (visited on 04/07/2026).
- [22] “FreeRTOS-Community-Supported-Demos/RISC-V_RV32_MPU_QEMU_VIRT_GCC at main · FreeRTOS/FreeRTOS-Community-Supported-Demos,” GitHub. (), [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Community-Supported-Demos/tree/main/RISC-V_RV32_MPU_QEMU_VIRT_GCC (visited on 04/07/2026).
- [23] “FreeRTOS scheduling (single-core, AMP and SMP) - FreeRTOS™,” FreeRTOS scheduling (single-core, AMP and SMP). (Mar. 2026), [Online]. Available: <https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/04-Task-scheduling> (visited on 04/07/2026).
- [24] S. Kadloor, N. Kiyavash, and P. Venkitasubramaniam, “Mitigating Timing Side Channel in Shared Schedulers,” *IEEE/ACM Transactions on Networking*, vol. 24, no. 3, pp. 1562–1573, Jun. 2016, ISSN: 1558-2566. DOI: 10.1109/TNET.2015.2418194. [Online]. Available: <https://ieeexplore.ieee.org/document/7089318/> (visited on 02/09/2026).
- [25] C.-Y. Chen, S. Mohan, R. Pellizzoni, R. B. Bobba, and N. Kiyavash, “A Novel Side-Channel in Real-Time Schedulers,” in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Apr. 2019, pp. 90–102. DOI: 10.1109/RTAS.2019.00016. [Online]. Available: <https://ieeexplore.ieee.org/document/8743170/> (visited on 02/09/2026).
- [26] “TCG Instruction Counting — QEMU documentation.” (), [Online]. Available: <https://www.qemu.org/docs/master/devel/tcg-icount.html> (visited on 04/28/2026).
- [27] “Cache Modelling TCG Plugin - QEMU.” (), [Online]. Available: <https://www.qemu.org/2021/08/19/tcg-cache-modelling-plugin/> (visited on 04/29/2026).